M. polireddy
192325036.

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

***CO1**: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3)*

Assessment Tool Weightage: 30%

Dr Dumpala Prasanth

QUESTIONS: 10 **Total Marks:50**

Annexure A

Constraint - Based Problem-Solving **Duration: 2Hrs**

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.

(10 Marks)

2. Implement an ε-greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ε values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.

(10 Marks)

3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.

(10 Marks)

4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.

(10 Marks)

5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

(10 Marks)

6. Implement a Reinforcement Learning agent that controls traffic signals at an intersection while ensuring that emergency vehicles receive immediate priority. Explain how exploration, exploitation, and policy learning are modified to satisfy the safety constraint.

(10 Marks)

7. Develop a Python program using Keras/TensorFlow to simulate a reinforcement learning agent for packet routing in a wireless network with limited bandwidth. Explain how bandwidth constraints influence the reward function and policy optimization.

(10 Marks)

8. Analyse an experimental comparison between random action selection and ε-greedy action selection in a Multi-Armed Bandit problem under a fixed time constraint. Record cumulative rewards and explain which strategy provides better performance.

(10 Marks)

9. Design a Reinforcement Learning framework for a delivery drone that must maximize successful deliveries while avoiding no-fly zones and maintaining battery limits. Explain the MDP formulation, Bellman equation, and policy learning process.

(10 Marks)

10. Implement a Reinforcement Learning solution using Python for a smart energy management system where electricity consumption must remain below a specified threshold. Explain how reward shaping, policy learning, and feasibility constraints improve decision-making.

(10 Marks)

Code of Conduct:

I M.Polireddy (192325056) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M Polireddy

RUBRICS FOR CALCULATION

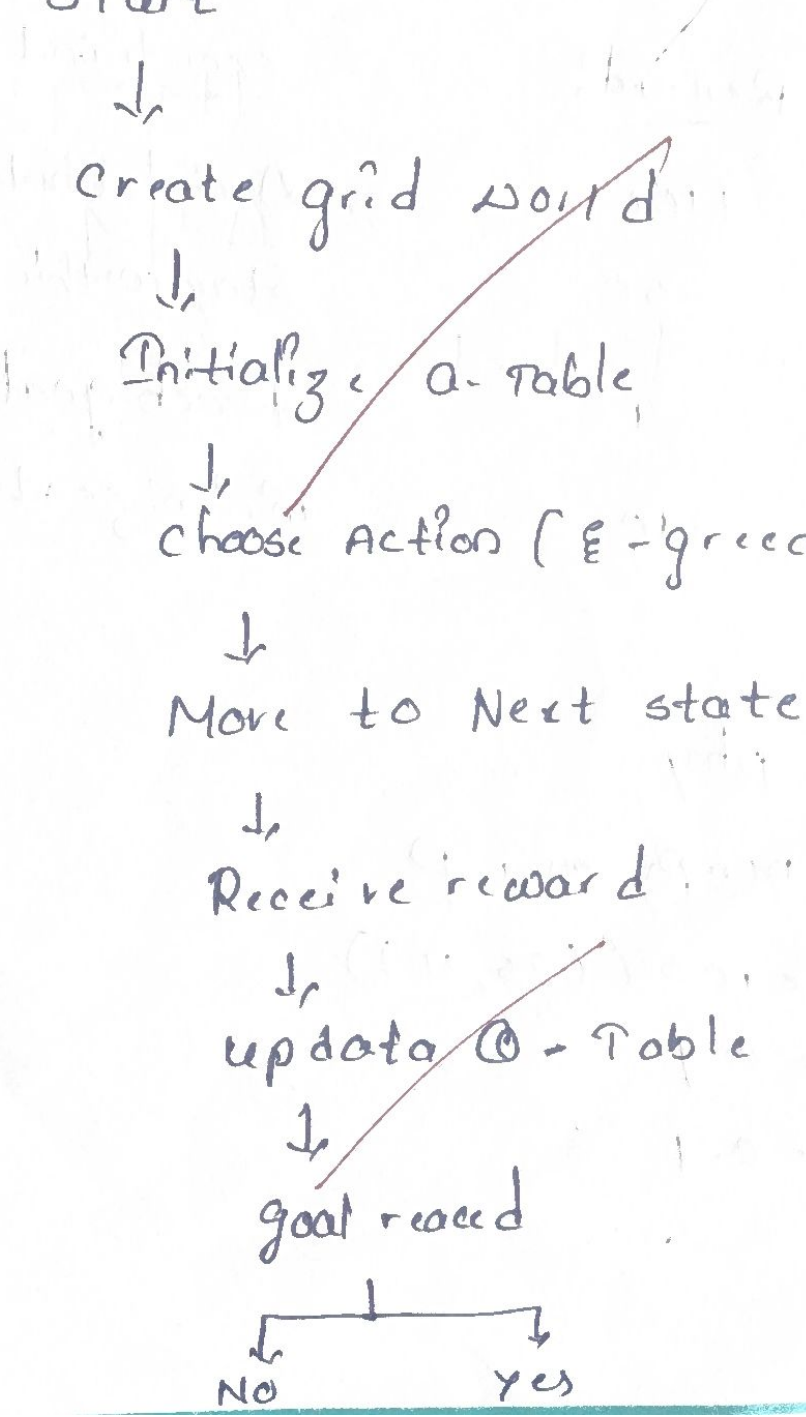
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Develop a python R.L Model using open AI grid world Navigation.

1. problem Understanding :-

The objective is to develop a (RL) agent that navigates through a grid world environment. the agent starts from the initial position and learn the optimal path to reach goal while avoiding obstacles the agent Improves the performance with the environment by Maximizing the cumulative reward.

Flow chart :-



Reinforcement learning component

Component:-

Environment

Agent

State space

Action space

Goal state

obstacle state

Description:-

5×5 grid world

RL Agent

each grid cell Represents one state

up, Down, Left, Right

Reach destination safely

Agent should Avoid these cells.

④ Reward function

Situation:- Reward:-

Reach Goal +100

Hit obstacle -50

normal More -1

Invalid more -10

⑤ constraints

constraint:- purpose:-

Avoid obstacles collision

stay within goals valid

Reach goal navigation

minimize steps efficiency

python Implementation:-

```
Impact phy
Import numpy as np
Q = np.Zeros((25, 4))
  alpha = 0.9
epsilon = 0.1
```

4. Working process :-

1. create grid world environment
2. Initialize the Q-Table with zero values
3. Perform the selected action
4. Receive reward from the environment
5. Receive reward action agent reaches the go
6. Store the learend policy for navigation.

5. Bellan equation :-

$$Q(s, a) = Q(s, a) + \alpha (R + \gamma \max Q(s', a') - Q$$

where :-

α = learing state

r = Discount factor

R = Reward

S = Current state

S' = Next state.

7. Conclusion :-

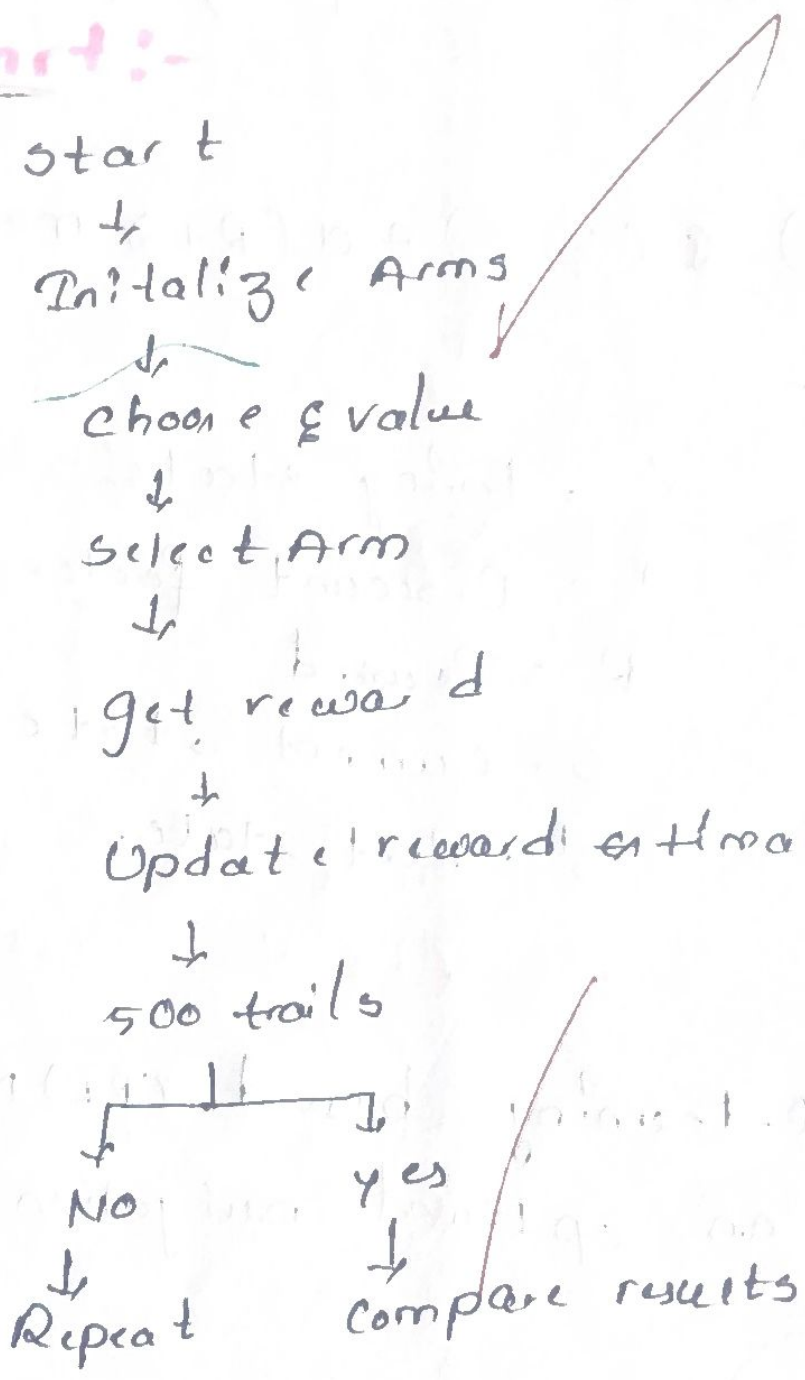
The Q-learning based (RL) Model succesful learns an optimal navigation policy.

Q:- Greedy Multi Armed Algorithm

Problem Statement:-

The ε greedy algorithm helps an agent maximize rewards help an agent Maximize rewards by balancing explanation erying new arms & exploitation choosing the best arm) with in 500 trails.

Flow chart:-



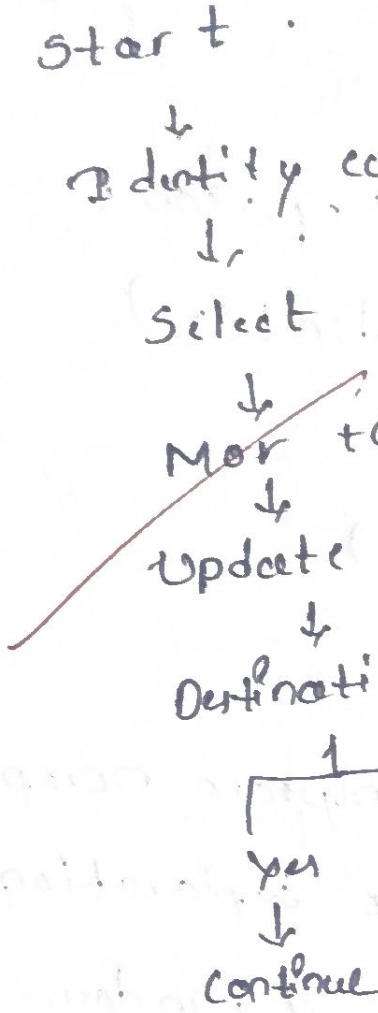
Parameters:-

parameters:-	value:-
Arms	5
Trails	500
Eralue	0.1; 0.3; 0.5

Problem statement :-

Design an MDP Model for an autonomous robot that reaches it destination while reducing energy usage and avoiding Unnecessary Movement.

Flow chart :-



Python syntax :-

```
states = ["start", "path", Goal"]
actions = ["up","Down", "left", "Right"]
Rewards = 100
battery_limit = 30;
```

syntax Meaning :-

states → Represents robot position
actions → possible Movement direction
Reward → Reward for reaching the goal.

Comparision:-

ε	exploration	performance
0.1	low	Best reward
0.3	reward	Balanced
0.5	High	lower Reward

Python Syntax:-

```
Import numpy as np
espilon =0.1
If np random.rand()< epsilon:
   action = np.random.Randint(5)
else():
   action = np.argmax(Q)
```

Syntax Meaning:-

* Import nump as np → Improre numpy library
* epsilon =0.1 → set the exploration rate
* np.random() → generates a random number.
* np.random Int() → selects a random arm.

Conclusion:-

The ε=0.1 policy provides the higest cumulativ reward by balancing limited exploration with medium exploration under the 500 trail constraint.

Q3: Design a Markov decision process (MDP) for an preferences

Value Iteration Algorithm for a grid Environment.

Problem statement:-
Implement a value iteration algorithm using python to find the optimal policy in a grid environment with Restricted states.

flowchart:-
Start -> Initialize -> Apply Bellman equation -> update s.value
-> check convergence -[No -> stop / yes -> continue.

Bellam equation:-
$V(s) = \max_a [R + \gamma V(s')]$

Python syntax:-
```
Import tensorflow as tf
Import numpy as np
gamma = 0.9
values = np.zeros ((5,5))
```

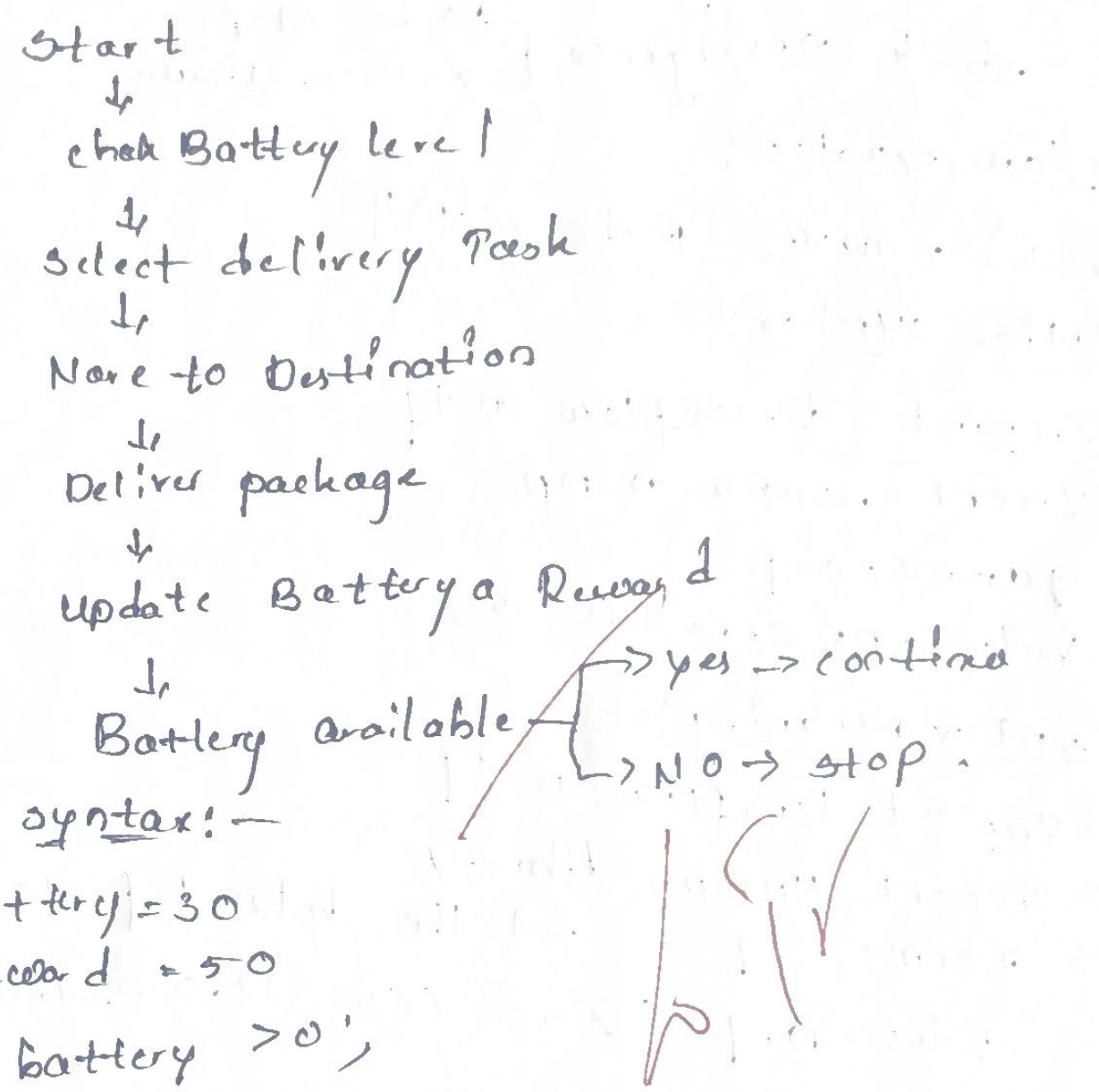
Syntax Meaning:-
-> Import tensflow as tf
-> Import Numpy library
-> gamma = 0.9 -> set the dataset factor
-> values = np.Zeros ((5,5)) -> Initializ. the values.

Conclusion:-
The value Iteration algorithm applies the Bellman equation to find the optimal policy while avoiding restricted states and maximizing long term rewards.

Q:- RL framework for a warehouse robot with Battery constraint.

Problem statement:-
Design a (RL) framework for warehouse robot that completes the max no of deliveries without exceeding a 30 minutes battery limit.

Flow chart:-



Python syntax:-

```
battery = 30
record = 50
If battery > 0':
  action = "deliver"
else:
  action = "Recharge":
```

Conclusion:-
The RL frame work helps the ware-house robot Maximize deliviers while staying with in 30 minutes battery limit ensuring efficient and reliable operation.